



FRONT-END-FRAMEWORK REACTJS



Tham khảo: frontend.daca.vn

NỘI DUNG

01

Multiple Page Application và Single Page Application

03

Hướng dẫn cài đặt

05

Cách ReactJS render nội dung HTML ra trình duyệt

02

Khái niệm ReactJS

04

Giải thích cấu trúc thư mục và ý nghĩa các file

06

Ôn tập một số lý thuyết Javascript (Hay dùng trong ReactJS)

01. MULTIPLE PAGE APPLICATION VÀ SINGLE PAGE APPLICATION

1.1. Multiple Page Application là gì?

- **MPA** là kiểu website hoạt động theo cách truyền thống, khi người dùng yêu cầu một trang web, thì server sẽ tính toán và trả về trang web đó cho người dùng toàn bộ trang web dưới dạng mã html và web đó sẽ được load lại mới hoàn toàn.
- Ví dụ: chonoo.vn, tiki.vn, sendo.vn



01. MULTIPLE PAGE APPLICATION VÀ SINGLE PAGE APPLICATION

1.2. Single Page Application là gì?

- **SPA** là một trang web mà ở đó người dùng có thể truy cập nhiều trang con mà không ảnh hưởng gì đến trang gốc. Khi người dùng truy cập vào thành phần bất kỳ trên trang, SPA sẽ chỉ chạy nội dung trên thành phần đó mà không tải lại toàn bộ trang như web truyền thống



02. KHÁI NIỆM REACTJS

- **ReactJS** là một mã nguồn mở được phát triển bởi **Facebook**, ra mắt vào năm **2013**.
- **ReactJS** là một thư viện viết bằng **Javascript**, dùng để xây dựng giao diện người dùng (**UI**).
- Giúp bạn xây dựng trang web **SPA (Single Page Application)** một cách nhanh chóng.
- Mức độ phổ biến của **ReactJS**: <https://trends.google.com/trends/?geo=VN>



React JS

03. HƯỚNG DẪN CÀI ĐẶT

- **Bước 1:** Cài đặt Node.js trước. Chỉ cần cài 1 lần duy nhất, nếu máy tính đã cài rồi thì không cần cài nữa.
- **Bước 2:** Gõ lệnh `npm install -g create-react-app`. Chỉ cần cài 1 lần duy nhất. Create-react-app là cách để đơn giản hóa quá trình cài đặt và cấu hình môi trường.
- **Bước 3:** Gõ lệnh `npx create-react-app ten-du-an`. Để tạo mới một dự án ReactJS.
- **Bước 4:** Gõ lệnh `cd ten-du-an`. Để đi vào thư mục dự án.
- **Bước 5:** Gõ lệnh `npm start`. Để khởi chạy dự án.

04. GIẢI THÍCH CẤU TRÚC THƯ MỤC VÀ Ý NGHĨA CÁC FILE

- **node_modules:** chứa các thư viện được cài đặt cho dự án, tất cả cài đặt sẽ được lưu tại đây, chúng ta không thao tác trong thư mục này.
- **public:** chứa tất cả file output, là các file sẽ tương tác trực tiếp với trình duyệt như: HTML, image, ...
- **src:** chứa tất cả các file input, đây là các file mà chúng ta sẽ quan tâm nhất, thao tác phần lớn ở những file này, gồm các file Javascript, CSS,...
- **package.json:** file này để lưu trữ thông tin (tên package, phiên bản, các dependencies) mà project của bạn sử dụng.
- **README.md:** file này để mô tả dự án, hướng dẫn cài đặt dự án do chính bạn tự viết ra.

Tìm hiểu cú pháp viết file Markdown tại <https://www.markdownguide.org/basic->

05. CÁCH REACTJS RENDER NỘI DUNG HTML RA TRÌNH DUYỆT

- Để hiểu rõ hơn về render trong React, ta xem **nội dung 3 file**:

↴ /public/index.html

↴ /src/App.js

↴ /src/index.js



05. CÁCH REACTJS RENDER NỘI DUNG HTML RA TRÌNH DUYỆT

- **/public/index.html**

- ↴ File này không chứa bất kỳ nội dung nào hiển thị ngoài trình duyệt.
- ↴ Trong file này ta chỉ cần quan tâm đến dòng `<div id="root"></div>`, tất cả HTML sau này được render vào trong `id="root"` này.

/src/App.js

- ↴ Đoạn code **bên trong return** chính là nội dung được hiển thị ngoài trình duyệt.
- ↴ File này liên kết với **/public/index.html** thông qua file **/src/index.js**
- ↴ Mở trình duyệt lên và bật F12 lên để xem nội dung HTML, code trong file App.js đã được render vào trong **id="root"**.

05. CÁCH REACTJS RENDER NỘI DUNG HTML RA TRÌNH DUYỆT

- **/src/index.js**

↴ Chúng ta quan tâm đến các đoạn code sau:

```
import App from './App';  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
  
root.render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
);
```

↴ Nội dung trên có nghĩa là file **/src/index.js** lấy nội dung từ **function App** của file **/src/App.js** render ra nội dung trả về **id="root"** của file **/public/index.html**, sau đó hiển thị nội dung này ra trình duyệt.

06. ÔN TẬP MỘT SỐ LÝ THUYẾT JAVASCRIPT

6.1. Variables (Biến)

- Có **3 cách khai báo giá trị cho một biến: var, let, const.**

↴ Khai báo giá trị với **var**

↴ **Khai báo toàn cục:** Sử dụng được trong toàn bộ chương trình.

↴ **Khai báo cục bộ:** Nếu khai báo trong hàm thì chỉ sử dụng được ở trong hàm.

↴ Khai báo giá trị với **let**

↴ **let** sử dụng như var, tuy nhiên có tác dụng phạm vi bên trong một khối (như bên trong câu điều kiện if, vòng lặp for, ...).

↴ Khai báo giá trị với **const**

↴ **const** chỉ mang duy nhất một giá trị, nếu giá trị thay đổi sẽ báo lỗi.

06. ÔN TẬP MỘT SỐ LÝ THUYẾT JAVASCRIPT

6.2. Default parameters (Tham số mặc định)

- Tham số mặc định cho phép các tham số mang giá trị mặc định nếu tham số không có giá trị hoặc giá trị không xác định (undefined).
- Hoặc có thể hiểu tham số mặc định là tham số ban đầu được gán cho function.
- Có 2 cách khai báo tham số mặc định: **gán mặc định tại vị trí khai báo và gán bên trong function.**

06. ÔN TẬP MỘT SỐ LÝ THUYẾT JAVASCRIPT

6.3. Spread syntax

- **Spread syntax** (Cú pháp trải ra) là một phép lặp lại các phần tử của mảng (array) hoặc đối tượng (object).
- Được thể hiện dưới dạng **dấu ba chấm: ...**

06. ÔN TẬP MỘT SỐ LÝ THUYẾT JAVASCRIPT

6.4. Rest parameters

- **Rest parameters (Tham số "còn lại")** là tham số đại diện cho những tham số không được khai báo của một function.
- Khi sử dụng khai báo đại diện bên trong một function thì khi gọi function sẽ không giới hạn giá trị truyền vào.
- Được ký hiệu bằng khai báo **...name** (cẩn thận nhầm với spread syntax - dùng trong array và object).

06. ÔN TẬP MỘT SỐ LÝ THUYẾT JAVASCRIPT

6.5. Destructuring

- **Destructuring** (phá vỡ cấu trúc) cho phép chúng ta dễ dàng sử dụng các giá trị phần tử của **Array** hoặc **Object** (Mỗi lần lấy giá trị sẽ ngắn hơn).



06. ÔN TẬP MỘT SỐ LÝ THUYẾT JAVASCRIPT

6.6. Arrow function

- **Cú pháp:**

```
var functionName = (val1, val2, ...) => {  
    /* Code */  
}
```


06. ÔN TẬP MỘT SỐ LÝ THUYẾT JAVASCRIPT

6.7. Export và Import

- **Cú pháp Export:**

```
export const functionName = () => {  
  /* Code */  
}
```

- **Cú pháp Import:**

```
import { functionName } from "./file_export.js";
```

Hoặc:

```
import * as functionName from "./file_export.js";
```

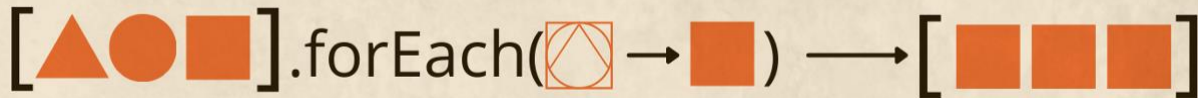
06. ÔN TẬP MỘT SỐ LÝ THUYẾT JAVASCRIPT

6.8. forEach()

- **forEach()** là một phương thức có sẵn của array, để duyệt qua mỗi phần tử của mảng và thực hiện một hành động nào đó.
- **forEach()** không tạo ra mảng mới, chỉ có thể sửa mảng hiện tại.
- Cú pháp:

```
arr.forEach(function(currentValue, index, array) {  
    // code  
});
```

- Trong đó:
 - ↳ **currentValue**: phần tử hiện tại đang được xử lý của array.
 - ↳ **index**: chỉ số của phần tử hiện tại đang được xử lý của array.
 - ↳ **array**: mảng hiện tại đang gọi hàm **forEach()**.

[triangle, circle, square].forEach(⬡ → ■) → [■, ■, ■]

06. ÔN TẬP MỘT SỐ LÝ THUYẾT JAVASCRIPT

6.9. map()

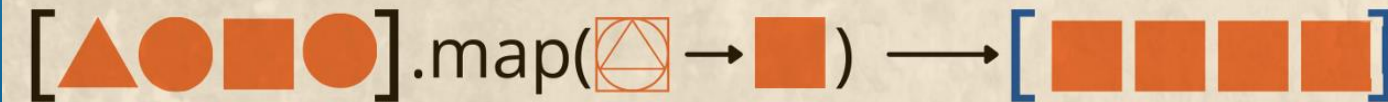
- **map()** sẽ duyệt qua từng phần tử của mảng. Giá trị trả về của hàm map là **một mảng mới**, với số lượng phần tử bằng với mảng cũ, nhưng giá trị của các phần tử thì được quyết định bởi lệnh return của hàm map().

- Cú pháp:

```
arr. map(function(currentValue, index, array) {  
    // code  
});
```

- Trong đó:

- ↳ **currentValue**: phần tử hiện tại đang được xử lý của array.
- ↳ **index**: chỉ số của phần tử hiện tại đang được xử lý của array.
- ↳ **array**: mảng hiện tại đang gọi hàm map().



NỘI DUNG

01

JSX

02

Components

03

Props

04

Hướng dẫn chèn icon

05

Events (Sự kiện)



01. JSX

- **JSX** viết tắt của từ **Javascript XML**
- Nó cho phép bạn **viết** các đoạn mã **HTML** trong **ReactJS** một cách dễ dàng và có cấu trúc hơn.

Mô tả	Cấu trúc HTML	Cấu trúc JSX
Tên Class	<tag class="">	<tag className="">
Thuộc tính value <input />	<input value="" />	<input defaultValue="" />
Thuộc tính for của <label>	<label for="">	<label htmlFor="">
Giá trị của <select><option>	<option value="">	<option value={}>
Style trực tiếp bên trong tag	<tag style="width: 10%">	<tag style={{ width: '10%' }}>
Event	<tag onclick="functionName()">	<tag onClick={functionName}>
Khi gọi một biến		 Hello {name}!

01. JSX

- **Lưu ý:**

- ↴ Trong JSX chỉ viết được 1 element cha bọc ở bên ngoài.
- ↴ Để viết được 2 element cha thì chúng ta dùng cú pháp **Fragment**.
- ↴ Cú pháp: `<></>`.

02. COMPONENTS

- **Components (Thành phần)** giúp phân chia các UI (giao diện người dùng) thành các phần nhỏ để dễ dàng quản lý và tái sử dụng.
- Ví dụ: header, footer, sidebar,...
- **Các bước tạo 1 component trong ReactJS**
 - ↳ **Bước 1:** Trong folder **src** tạo một folder mới tên là **components**.
 - ↳ **Bước 2:** Trong folder **components** tạo một folder mới đặt tên theo đúng ý nghĩa của khối đó. Ví dụ header đặt tên folder là Header.
 - ↳ **Bước 3:** Tạo 1 file mới đặt tên là **index.js**. Sau đó viết 1 function tên là Header và export default.
 - ↳ **Bước 4: Import** vào file nào mà bạn muốn sử dụng component đó.

02. COMPONENTS

- **Ví dụ:** Dựng layout sau bằng cách chia các component. Sau đó import cả CSS vào cho



03. PROPS

- **Props là một object** được truyền vào trong một component.
- **Props** cho phép chúng ta **giao tiếp giữa các components với nhau bằng cách truyền tham số** qua lại giữa các components.
- Khi một components cha truyền cho component con một props thì **components con chỉ có thể đọc** và không có quyền chỉnh sửa nó bên phía components cha.
- Cách truyền một props cũng giống như cách mà bạn thêm một attributes cho một element HTML.
- **Props có thể nhận giá trị là tất cả các kiểu dữ liệu:**
 - ↳ Kiểu dữ liệu nguyên thủy: String, Number, Boolean, Undefined, Null, Symbol
 - ↳ Kiểu dữ liệu phức tạp: Function, Object (Object, Array)

04. HƯỚNG DẪN CHÈN ICON

- Cài đặt thêm package (gói): <https://www.npmjs.com/package/react-icons>
- Sử dụng câu lệnh: **npm i react-icons**
- Link trang chủ: <https://react-icons.github.io/react-icons>
- Cách chèn icon vào React

↴ **Bước 1:** Import icon vào file như sau:

```
import { IoSearchOutline } from "react-icons/io5";
```

↴ **Bước 2:** Chèn icon vào đoạn code muốn hiển thị:

```
<IoSearchOutline />
```

05. EVENTS (SỰ KIỆN)

- **Xử lý các sự kiện** trong React rất **giống** với xử lý các sự kiện trong **Javascript**, nhưng có một số khác biệt về cú pháp.
- Các sự kiện trong React được đặt tên bằng **camelCase**, thay vì chữ thường.
- Một số sự kiện phổ biến trong ReactJS:
 - ↳ `onClick`
 - ↳ `onChange`
 - ↳ `onSubmit`
 - ↳ `onFocus`
 - ↳ `onBlur`

NỘI DUNG

01

Conditional Rendering (Render với điều kiện)

02

Lists

03

Sử dụng CSS/SCSS trong ReactJS

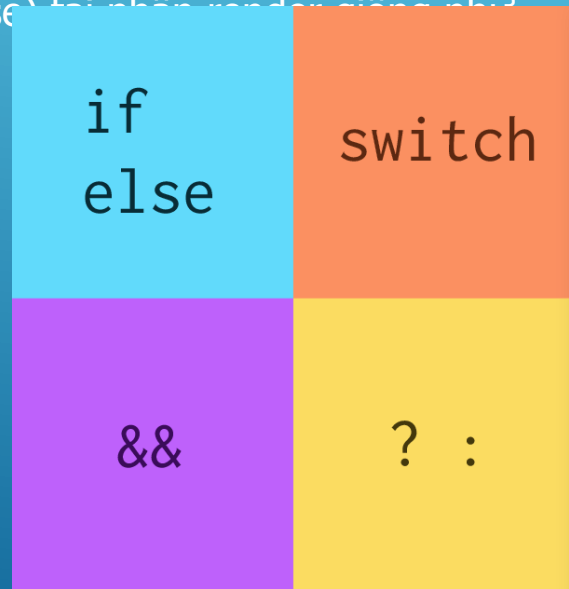
04

Hooks là gì?

A series of white diagonal lines of varying lengths and thicknesses, located in the bottom right corner of the slide.

01. CONDITIONAL RENDERING (RENDER VỚI ĐIỀU KIỆN)

- Trong React, chúng ta có thể tạo nhiều component khác nhau, khi đó có thể render bất kỳ component nào ta muốn, bằng cách sử dụng điều kiện tại phần render.
- Cách sử dụng câu điều kiện (câu điều kiện if else) tương tự như cách sử dụng trong Javascript.
- Ví dụ 1: Tính năng login, logout.
- Ví dụ 2: Cú pháp &&
(Ví dụ sẽ demo trong buổi học)



02. LISTS

- Ví dụ 1: Từ một mảng menu in ra giao diện

```
const arrayMenu = [  
    "Trang chủ",  
    "Sản phẩm",  
    "Tin tức",  
    "Giới thiệu",  
    "Liên hệ",  
];
```

- Trang chủ
- Sản phẩm
- Tin tức
- Giới thiệu
- Liên hệ

02. LISTS

- Ví dụ 2: Truyền props từ component cha là ProductList/index.js vào component con là ProductItem.js
- Ví dụ 3: List dạng lồng nhau. Có một mảng country, trong mảng country có mảng con là city. Lặp qua từng phần tử của mảng country, sau đấy lặp qua từng phần tử của mảng city và vẽ ra giao diện.

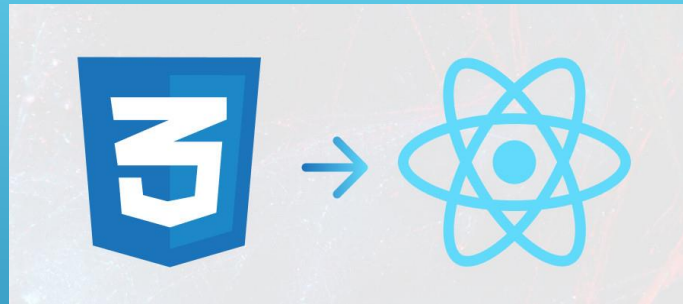
(Ví dụ sẽ demo trong buổi học)

03. SỬ DỤNG CSS/SCSS TRONG REACTJS

3.1. Sử dụng CSS trong ReactJS

- **Các bước sử dụng CSS trong ReactJS:**
 - **Bước 1:** Tạo 1 file CSS ở cùng cấp với component.
 - **Bước 2:** Nhúng file CSS đó vào trong component bằng cú pháp: `import './TenFile.css';`
- **Ví dụ:** Lấy ví dụ phần 2 để CSS lại cho đẹp.

Viet Nam
Ha Noi
Da Nang
Ho Chi Minh
Thai Lan
Bang Coc
Phuket



NỘI DUNG

01

useState

02

useEffect



01. USESTATE

1.1. Khái niệm

- **useState** giúp **cập nhật lại trạng thái** của **dữ liệu** (hay cập nhật lại giá trị của dữ liệu).
- Khi **dữ liệu thay đổi** thì **giao diện được cập nhật lại** theo dữ liệu mới.
- Một vài **ví dụ** trong thực tế:
 - Bóng đèn có 2 trạng thái là bật hoặc tắt.
 - Trạng thái đã đăng nhập hoặc chưa đăng nhập vào tài khoản.
 - Khi tăng số lượng sản phẩm thì tổng tiền được cập nhật lại.

01. USESTATE

1.1. Khái niệm

- **Cú pháp:**

```
import { useState } from "react";  
  
function Component() {  
  const [state, setState] = useState(initialStateValue);  
  // Code  
}  
  
export default Component;
```

- **Trong đó:**

- **state:** là tên biến của state.
- **setState:** là một function dùng để cập nhật state.
- **initialStateValue:** là giá trị khởi tạo (ban đầu) của state, chỉ dùng 1 lần.

01. USESTATE

1.2. Ví dụ

- **Ví dụ 1:** Bóng đèn có 2 trạng thái là bật hoặc tắt.
- **Ví dụ 2:** Khi tăng số lượng sản phẩm thì tổng tiền được cập nhật lại.
- **Ví dụ 3:** Tạo gợi ý Search giống của Youtube sử dụng onChange và fake data.

02. USEEFFECT

2.1. Khái niệm

- **useEffect** dùng để **xử lý logic** nào đó **khi data** (dữ liệu) được **thay đổi**.
- Component sau khi được render ra giao diện lần đầu, thì sẽ gọi tới hàm callback của `useEffect` (vì chúng ta ưu tiên việc render ra giao diện trước, xử lý logic sau nên callback của `useEffect` sẽ chạy sau khi component được render ra).
- **Cú pháp:**
`useEffect(callback, [dependency]);`
- **Trong đó:**
 - **callback:** làm hàm gọi lại, bắt buộc phải có.
 - **dependency:** là một biến. Không bắt buộc phải có.

02. USEEFFECT

2.2. useEffect(callback)

- Khi **render lại giao diện** (tức render lần 2 trở đi), thì **callback** của `useEffect` **vẫn được gọi lại**.
- **Ví dụ:** `querySelectorAll` các item trong DOM.

02. USEEFFECT

2.3. useEffect(callback, [])

- Khi **render lại giao diện** (tức render lần 2 trở đi), thì **callback** của useEffect **không được gọi lại**.
- Thường áp dụng cho gọi API 1 lần để lấy dữ liệu.
- **Ví dụ:** Lấy data từ api và render ra giao diện.

02. USEEFFECT

2.4. useEffect(callback, [dependency])

- Khi **render lại giao diện** (tức render lần 2 trở đi), thì **callback** của useEffect **được gọi lại khi dependency thay đổi**.
- **Ví dụ:** Làm pagination (phân trang).

Api dữ liệu mẫu: <https://dummyjson.com/products?skip=0&limit=30>

A series of white diagonal lines of varying lengths and positions, located in the bottom right corner of the slide, serving as a decorative element.

03. SỬ DỤNG CSS/SCSS TRONG REACTJS

3.2. Sử dụng SCSS trong ReactJS

- **Các bước sử dụng SCSS trong ReactJS:**
 - **Bước 1:** Gõ lệnh `npm i sass` (Link trang NPM: <https://www.npmjs.com/package/sass>) để cài SASS cho project và chỉ cần cài 1 lần duy nhất cho project.
 - **Bước 2:** Tạo 1 file SCSS ở cùng cấp với component.
 - **Bước 3:** Nhúng file SCSS đó vào trong component bằng cú pháp: `import './TenFile.scss';`
- Ví dụ: Vẫn ví dụ trên nhưng code bằng SCSS.



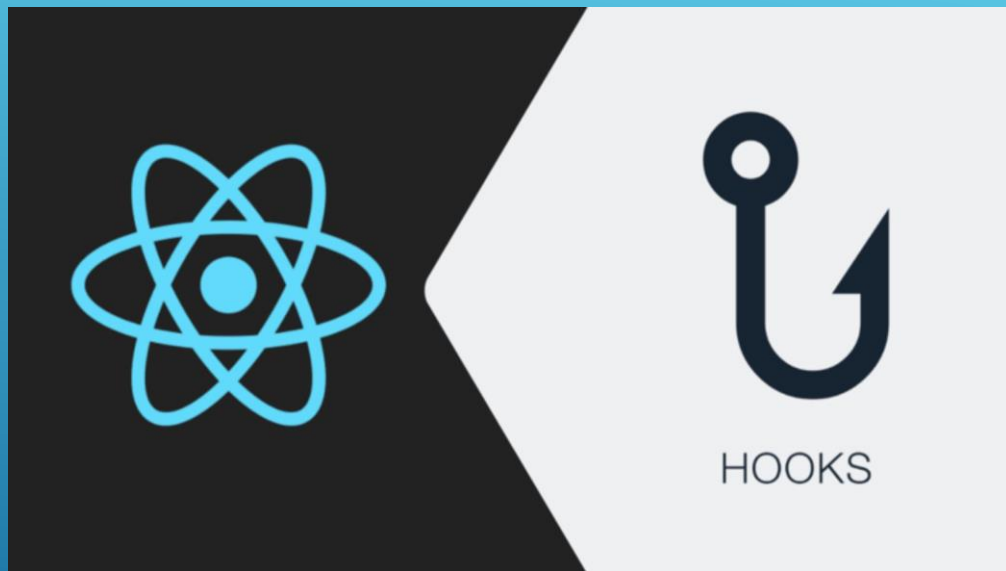
04. HOOKS LÀ GÌ?

- **Hooks** (gắn vào, móc vào)
- Có **2 loại component**: class component và **function component**
- Trước đây, class component có đầy đủ tính năng còn function component thiếu khá nhiều tính năng, nên người ta thường code theo hướng class component.
- **Hooks** mới được thêm ở phiên bản React 16.8. Hooks **bổ sung** thêm những **tính năng còn thiếu cho function component** để có đầy đủ tính năng giống như class component.
- Chính vì vậy, ngày nay người ta thường code theo hướng function component vì tính năng đã đầy đủ mà cú pháp ngắn hơn, dễ hiểu hơn nhiều so với class component.
- Hooks bản chất là những cái hàm được viết sẵn trong ReactJS được sử dụng để code các tính năng khác nhau, để sử dụng được các tính năng này ở trong các component ta cần gắn các hooks này vào trong component

04. HOOKS LÀ GÌ?

- Có **10 loại hooks**:

- useState
- useEffect
- useContext
- useRef
- useCallback
- useMemo
- useReducer
- useEffect
- useImperativeHandle
- useDebugValue



NỘI DUNG

01

`useContext`

03

`React.memo()`

02

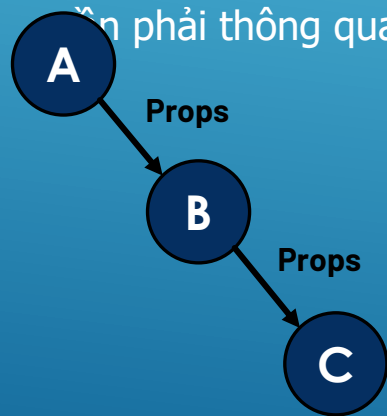
`useRef`

04

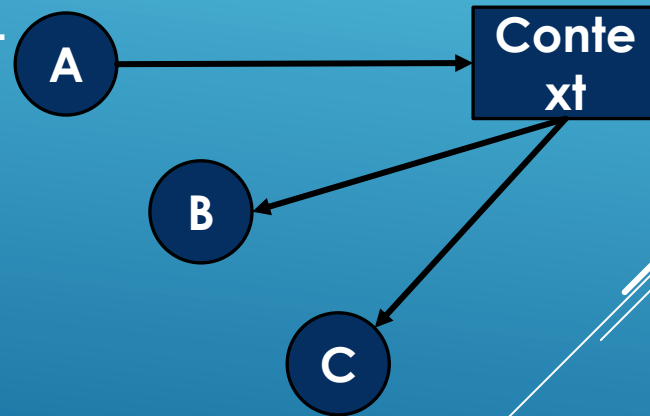
`useCallback`

01. USECONTEXT

- **useContext** (bối cảnh) giúp **đơn giản hóa việc chuyển dữ liệu** từ component cha xuống các component con mà không cần phải sử dụng đến props.
- Tức là chuyển trực tiếp từ component cha xuống các component con mà không cần phải thông qua 1 component gián tiếp.



Cách truyền data từ component cha xuống component con sử dụng props.



Cách truyền data từ component cha xuống component con sử dụng useContext.

01. USECONTEXT

- Các bước để sử dụng useContext:
 - **Bước 1:** Tạo ra 1 bối cảnh ở component A (*để tạo ra phạm vi và sử dụng được data trong phạm vi đó, ví dụ phạm vi là component A thì tất cả các component con đều sử dụng được*).

```
import { createContext } from "react";  
export const AContext = createContext();
```

01. USECONTEXT

- Các bước để sử dụng useContext:
 - **Bước 2:** Cung cấp bối cảnh để bao bọc toàn bộ các component cần sử dụng data.

```
function A() {  
  return (  
    <AContext.Provider value={data}>  
      <B />  
    </AContext.Provider>  
  );  
}
```

01. USECONTEXT

- Các bước để sử dụng useContext:
 - **Bước 3:** Sử dụng useContext ở component con để có thể lấy được data từ component cha (Ví dụ component C là con của component B).

```
import { useContext } from "react";
import { AContext } from "url_component_A";
function C() {
  const data = useContext(AContext);
  return (
    <>
      {data}
    </>
  );
}
```


02. USEREF

- **useRef** trả về một object với thuộc tính `current` được khởi tạo thông qua tham số truyền vào.
- Object được trả về không bị khởi tạo lại khi component render lại.
- Giá trị trong object thay đổi nhưng component không bị render lại (`useState` thay đổi thì làm component render lại).
- Cú pháp: **`const tenBien = useRef(initialValue);`**
- Trong đó:
 - **`initialValue`**: là giá trị khởi tạo.
- **useRef** được sử dụng để truy cập được các phần tử trong DOM.

03. REACT.MEMO()

- **React.memo()** dùng để **ghi nhớ kết quả**.
- React.memo() sẽ không render lại component khi component đó không có gì thay đổi.
- React.memo() là Higher Order Component (Thành phần bậc cao hơn) được sử dụng để bọc các component.

- **Cú pháp:**

```
import { memo } from "react";  
function Component() {  
  return (  
    <></>  
  )  
}  
export default memo(Component);
```

04. USECALLBACK

- **useCallback** giúp tránh thực hiện lại một hàm không cần thiết.
- Giúp tạo ra một vùng nhớ để lưu hàm callback và chỉ tạo ra hàm callback mới khi dependencies thay đổi.
- Cú pháp: **const functionName = useCallback(callback, dependency);**
- Trong đó:
 - **callback**: là một hàm được gọi lại, lần render đầu tiên luôn chạy vào hàm này.
 - **dependency**: là sự phụ thuộc, khi dependency thay đổi thì useCallback mới tạo ra một hàm mới.